Benemérita Universidad Autónoma de Puebla

Facultad de Ciencias de la Computación

Ingeniería en Tecnologías de la Información

Materia
Minería de Datos

Reporte

Práctica 4: Clasificación mediante Redes Neuronales

Profesor:

Pedro T. Vera

Integrantes:

Blanca Flor Visca Cocotzin
Christopher Aguilar Mendez
Angel Fernando Ruiz Vasquez

24 de marzo de 2026

Índice

1. Introducción **3**

2. Descripción del algoritmo: Redes Neuronales Artificiales **4**
2.1. Estructura de una red neuronal multicapa (MLP) 4
2.2. Funcionamiento del neurón artificial 4
2.3. Funciones de activación 5
2.4. Proceso de entrenamiento 5
2.5. Funciones de pérdida 6
2.6. Algoritmo de optimización: Gradiente Descendente 6
2.7. Descripción de la base de datos 7
2.8. Fórmulas y métricas utilizadas 8

3. Metodología **9**

4. Pruebas y Resultados **10**
4.1. Topología de la red neuronal propuesta 10

5. Resultados obtenidos **10**
5.1. Evaluación del modelo 10
5.2. Predicción de nuevos datos 11

6. Conclusiones **12**

7. Apéndices **13**

Minería de Datos
Práctica 4: Clasificación mediante Redes Neuronales

Blanca Flor Visca Cocotzin, Christopher Aguilar Mendez, Angel Fernando Ruiz Vasquez

24 de marzo de 2026

Resumen

En esta práctica se abordó el problema de clasificación de vinos mediante un modelo de redes neuronales artificiales, utilizando la base de datos *Wine* del repositorio UCI. La base de datos contiene 178 instancias descritas por 13 atributos químicos continuos, distribuidas en tres clases correspondientes a diferentes cultivares de vino producidos en la misma región de Italia. Se utilizó el software KNIME Analytics Platform para configurar, entrenar y evaluar una red neuronal multicapa (*Multilayer Perceptron*), proponiendo una topología adecuada y ajustando parámetros como el número de capas ocultas, la función de activación, el algoritmo de optimización y la tasa de aprendizaje. El desempeño del modelo se evaluó mediante la matriz de confusión y las métricas de exactitud, precisión, sensibilidad y tasa de error sobre el conjunto de prueba, y finalmente se realizó la clasificación de datos nuevos.

1. Introducción

Las redes neuronales artificiales constituyen uno de los modelos más versátiles y potentes dentro del aprendizaje automático, inspiradas en el funcionamiento del sistema nervioso biológico. Su capacidad para aprender relaciones complejas y no lineales entre variables las convierte en una herramienta ampliamente utilizada en tareas de clasificación, regresión y reconocimiento de patrones.

En esta práctica se aborda el problema de clasificación de vinos utilizando la base de datos *Wine* del repositorio UCI, la cual contiene los resultados de un análisis químico de vinos producidos en la misma región de Italia pero derivados de tres cultivares diferentes. Mediante la configuración y entrenamiento de una red neuronal multicapa (*Multilayer Perceptron*) en el software KNIME, se busca construir un modelo capaz de clasificar correctamente las muestras de vino a partir de sus 13 atributos químicos.

2. Descripción del algoritmo: Redes Neuronales Artificiales

Una **red neuronal artificial** (RNA) es un modelo matemático inspirado en sistemas biológicos, adaptado y simulado en computadoras convencionales. Según Haykin (1999), es un procesador masivamente paralelo y distribuido hecho de unidades procesadoras simples, propensas a almacenar conocimiento adquirido de la experiencia. Sus características principales son: **aprendizaje** (puede modificar su comportamiento en respuesta al medio ambiente), **generalización** (insensible a pequeños cambios en las entradas) y **abstracción** (puede determinar las características principales de un conjunto de datos).

2.1. Estructura de una red neuronal multicapa (MLP)

Un *Multilayer Perceptron* (MLP) se compone de tres tipos de capas:

1. **Capa de entrada:** recibe los datos de entrada. El número de neuronas corresponde al número de atributos del conjunto de datos. En este caso, 13 neuronas (una por cada atributo químico del vino).

2. **Capas ocultas (*hidden layers*):** procesan la información mediante combinaciones no lineales de las entradas. El número de capas ocultas y el número de neuronas en cada una constituyen la **topología** de la red, la cual debe ser propuesta por el diseñador.

3. **Capa de salida:** produce la predicción final. Para problemas de clasificación multiclase, el número de neuronas corresponde al número de clases. En este caso, 3 neuronas (clases 1, 2 y 3).

2.2. Funcionamiento del neurón artificial

El neurón artificial es un elemento procesador al que se aplica un conjunto de entradas. Las entradas x_i se multiplican por un peso w_i, las multiplicaciones se suman obteniendo el valor **net**, y este resultado se filtra por una función de activación:

$$net = \sum_{i=1}^{n} w_i \cdot x_i + b \tag{1}$$

$$a = f(net) \tag{2}$$

donde:

- x_i = valores de entrada.

- w_i = pesos sinápticos asociados a cada entrada.
- b = sesgo (*bias*).
- f = función de activación.
- a = salida de la neurona (activación).

2.3. Funciones de activación

Las funciones de activación introducen no linealidad en el modelo, permitiendo que la red aprenda relaciones complejas. Las más utilizadas son:

1) Sigmoide:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \tag{3}$$

2) Tangente hiperbólica (tanh):

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \tag{4}$$

3) ReLU (*Rectified Linear Unit*):

$$\text{ReLU}(z) = \text{máx}(0, z) \tag{5}$$

4) Softmax (utilizada en la capa de salida para clasificación multiclase):

$$\text{Softmax}(z_j) = \frac{e^{z_j}}{\sum_{k=1}^{K} e^{z_k}} \tag{6}$$

donde K es el número total de clases.

2.4. Proceso de entrenamiento

El entrenamiento de una red neuronal consiste en ajustar los pesos y sesgos para minimizar una **función de pérdida** que mide la diferencia entre las predicciones y los valores reales. El proceso sigue estos pasos:

1. **Propagación hacia adelante (*forward propagation*):** los datos de entrada se propagan a través de las capas de la red, generando una predicción.

2. **Cálculo de la pérdida:** se evalúa el error entre la predicción y la clase real mediante la función de pérdida.

3. **Retropropagación (*backpropagation*):** se calculan los gradientes de la función de pérdida respecto a cada peso, propagando el error hacia atrás desde la capa de salida.

4. **Actualización de pesos:** se ajustan los pesos utilizando un algoritmo de optimización (como el descenso del gradiente).

5. **Iteración:** los pasos anteriores se repiten durante múltiples épocas (*epochs*) hasta que la red converge o se alcanza un criterio de parada.

2.5. Funciones de pérdida

La función de pérdida (o función de error/costo) mide qué tan cercanas son las predicciones al conjunto de datos real. Para problemas de regresión se utiliza el **Error Cuadrático Medio** (MSE):

$$MSE = \frac{1}{m}\sum_{\mu=1}^{m}(\hat{y}_\mu - y_\mu)^2 \tag{7}$$

Para problemas de clasificación multiclase, se utiliza la **entropía cruzada categórica**:

$$L = -\sum_{i=1}^{N}\sum_{j=1}^{K} y_{ij} \cdot \log(\hat{y}_{ij}) \tag{8}$$

donde:

- N = número de instancias.
- K = número de clases.
- y_{ij} = valor real (1 si la instancia i pertenece a la clase j, 0 en caso contrario).
- $\hat{y}_{ij}$ = probabilidad predicha por el modelo.

2.6. Algoritmo de optimización: Gradiente Descendente

El método del **gradiente descendente** es un algoritmo de optimización fundamental para entrenar redes neuronales. El gradiente es la generalización vectorial de la derivada: indica hacia

dónde está el incremento de la función. Para minimizar el error, se toma el gradiente con signo contrario, moviéndose en la dirección que disminuye la función de pérdida:

$$w_{nuevo} = w_{actual} - \eta \cdot \frac{\partial E}{\partial w} \tag{9}$$

donde η es la **tasa de aprendizaje** (*learning rate*), un hiperparámetro que controla el tamaño de los ajustes en cada iteración. Los valores que se busca modificar para minimizar el error son los pesos w_i y los sesgos b.

2.7. Descripción de la base de datos

La base de datos *Wine* fue obtenida del repositorio UCI Machine Learning y contiene los resultados de un análisis químico de vinos cultivados en la misma región de Italia, derivados de tres cultivares diferentes. Contiene **178 instancias** descritas por **13 atributos numéricos continuos** y una variable de clase.

Tabla 1: Descripción de los atributos de la base de datos Wine

No.	**Atributo**	**Tipo**
1	Alcohol	Continuo
2	Malic acid	Continuo
3	Ash	Continuo
4	Alcalinity of ash	Continuo
5	Magnesium	Continuo
6	Total phenols	Continuo
7	Flavanoids	Continuo
8	Nonflavanoid phenols	Continuo
9	Proanthocyanins	Continuo
10	Color intensity	Continuo
11	Hue	Continuo
12	OD280/OD315 of diluted wines	Continuo
13	Proline	Continuo

La distribución de clases es la siguiente:

- **Clase 1:** 59 instancias (33.15 %).
- **Clase 2:** 71 instancias (39.89 %).
- **Clase 3:** 48 instancias (26.97 %).

Cabe destacar que la base de datos **no contiene valores faltantes** y todos los atributos son numéricos continuos, lo cual facilita el procesamiento directo por parte de la red neuronal. La base de datos se encuentra particionada en un archivo Excel con las siguientes hojas:

- **ALL:** conjunto completo con 178 instancias.
- **TRAINED:** conjunto de entrenamiento con 160 instancias (53 de clase 1, 65 de clase 2, 42 de clase 3).
- **TEST:** conjunto de prueba con 18 instancias (6 por cada clase).
- **NEW:** conjunto de datos nuevos con 15 instancias cuya clase es desconocida (marcada como "?"), a las cuales se les debe asignar una clase mediante el modelo entrenado.

2.8. Fórmulas y métricas utilizadas

Sea la matriz de confusión para un problema multiclase con K clases:

Real / Predicho	Clase 1	Clase 2	Clase 3
Clase 1	c_{11}	c_{12}	c_{13}
Clase 2	c_{21}	c_{22}	c_{23}
Clase 3	c_{31}	c_{32}	c_{33}

(10)

A partir de esta matriz se calculan las siguientes métricas:

1) Exactitud (Accuracy): proporción de predicciones correctas sobre el total:

$$Accuracy = \frac{\sum_{i=1}^{K} c_{ii}}{\sum_{i=1}^{K} \sum_{j=1}^{K} c_{ij}} \tag{11}$$

2) Tasa de error: proporción de clasificaciones incorrectas:

$$Error = 1 - Accuracy \tag{12}$$

3) Sensibilidad (Recall) por clase i:

$$Recall_i = \frac{c_{ii}}{\sum_{j=1}^{K} c_{ij}} \tag{13}$$

4) Precisión (Precision) por clase i:

$$Precision_i = \frac{c_{ii}}{\sum_{j=1}^{K} c_{ji}} \tag{14}$$

3. Metodología

Para el desarrollo de la práctica se siguió una metodología estructurada por etapas:

1. **Lectura de datos:** se cargó la base de datos *Wine* en KNIME desde el archivo `BDD-WINE.xlsx`, el cual contiene cuatro hojas: `ALL` (178 instancias), `TRAINED` (160 instancias para entrenamiento), `TEST` (18 instancias para evaluación) y `NEW` (15 instancias con clase desconocida).

2. **Análisis de atributos:** se identificaron los 13 atributos numéricos continuos y la variable de clase con tres valores posibles (1, 2, 3). Se verificó la ausencia de valores faltantes.

3. **Preprocesamiento:** se aplicó normalización a los atributos de entrada para escalarlos a un rango uniforme, mejorando la convergencia del entrenamiento de la red neuronal.

4. **Diseño de la topología de la red:** se propuso la arquitectura del *Multilayer Perceptron*, definiendo:

   - Número de neuronas en la capa de entrada: 13 (una por atributo).
   - Número y tamaño de las capas ocultas.
   - Número de neuronas en la capa de salida: 3 (una por clase).
   - Función de activación.
   - Algoritmo de optimización.
   - Tasa de aprendizaje.
   - Número de épocas de entrenamiento.

5. **Entrenamiento del modelo:** se entrenó la red neuronal utilizando el conjunto de entrenamiento, ajustando iterativamente los pesos mediante retropropagación.

6. **Evaluación del modelo:** se aplicó el modelo entrenado sobre el conjunto de prueba, obteniendo la matriz de confusión y calculando las métricas de exactitud, precisión, sensibilidad y tasa de error.

7. **Clasificación de datos nuevos:** se utilizó el modelo entrenado para asignar una clase a los objetos del conjunto *new*, mostrando las predicciones obtenidas.

4. Pruebas y Resultados

4.1. Topología de la red neuronal propuesta

La topología propuesta para la red neuronal fue la siguiente:

- **Capa de entrada:** 13 neuronas.
- **Capas ocultas:**
- **Capa de salida:** 3 neuronas (Softmax).
- **Función de activación:**
- **Algoritmo de optimización:**
- **Tasa de aprendizaje:**
- **Número de épocas:**
- **Función de pérdida:**

5. Resultados obtenidos

5.1. Evaluación del modelo

A partir del conjunto de prueba (`test`), se obtuvo la siguiente matriz de confusión:

Predicción / Real	1	2	3
1	6	0	0
2	0	6	0
3	0	0	6

Tabla 2: Matriz de confusión del modelo.

De acuerdo con los resultados:

- Instancias correctamente clasificadas: 18

- Instancias incorrectas: 0
- Exactitud (Accuracy): 100 %
- Tasa de error: 0 %

Instancia	Clase real	Clase predicha
1	1	1
2	1	1
3	1	1
4	1	1
5	1	1
6	1	1
7	2	2
8	2	2
9	2	2
10	2	2
11	2	2
12	2	2
13	3	3
14	3	3
15	3	3
16	3	3
17	3	3
18	3	3

Tabla 3: Comparación entre la clase real y la clase predicha en el conjunto de prueba.

Interpretación:

El modelo logró clasificar correctamente todas las instancias del conjunto de prueba. La matriz de confusión muestra que no existen errores de clasificación, lo cual indica que la red neuronal aprendió de manera adecuada los patrones presentes en los datos.

Este comportamiento puede atribuirse a la buena separabilidad entre las clases del conjunto de datos.

5.2. Predicción de nuevos datos

El modelo entrenado fue aplicado al conjunto de datos `new`, el cual no contiene etiquetas reales. Por lo tanto, su propósito es generar predicciones para nuevas instancias.

Instancia	Clase predicha
1	2
2	2
3	2
4	2
5	2
6	3
7	3
8	3
9	3
10	3
11	1
12	1
13	1
14	1
15	1

Tabla 4: Clasificación de los datos nuevos.

Interpretación:

El modelo asignó una clase a cada una de las instancias del conjunto `new`. Estas predicciones representan el comportamiento del modelo en un escenario real, donde no se conocen las etiquetas verdaderas.

Se observa una distribución equilibrada entre las tres clases, lo que indica que el modelo es capaz de generalizar adecuadamente a partir de los datos de entrenamiento.

6. Conclusiones

En esta práctica se implementó exitosamente un modelo de clasificación basado en redes neuronales artificiales utilizando la base de datos *Wine* del repositorio UCI, la cual ya se encontraba previamente particionada en conjuntos de entrenamiento, prueba y datos nuevos.

El modelo empleado, basado en un perceptrón multicapa (Multilayer Perceptron), alcanzó una exactitud del 100 % sobre el conjunto de prueba, lo cual indica que fue capaz de aprender correctamente los patrones presentes en los datos. Este resultado refleja una adecuada separabilidad entre las clases, permitiendo al modelo realizar clasificaciones sin errores.

La matriz de confusión confirmó este comportamiento, mostrando que todas las instancias fueron clasificadas correctamente en sus respectivas clases, sin presentar errores de predicción.

Asimismo, la clasificación de los datos nuevos demostró que el modelo entrenado es capaz de generalizar el conocimiento adquirido y aplicarlo a instancias no etiquetadas, asignando una clase a cada registro. Esto representa un escenario real donde el modelo se utiliza para predecir información desconocida.

Finalmente, la práctica permitió reforzar la comprensión del uso de redes neuronales en problemas de clasificación, así como la importancia del preprocesamiento de datos, la correcta separación de conjuntos y el uso de herramientas como KNIME para la construcción y evaluación de modelos de aprendizaje automático.

7. Apéndices

Los siguientes apéndices se entregan como documentos independientes:

A) Código fuente / configuración del modelo de red neuronal.

B) Marco teórico: conceptos de redes neuronales (documento `Marco_Teorico.pdf`).

C) Manual de usuario: procedimiento en KNIME.

D) Referencias bibliográficas (incluidas en este documento).

E) Manual técnico.

Referencias

[1] Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press.

[2] Han, J., Kamber, M., and Pei, J. (2012). *Data Mining: Concepts and Techniques*. Morgan Kaufmann, 3 edition.

[3] Haykin, S. (2009). *Neural Networks and Learning Machines*. Pearson, 3 edition.

[4] KNIME AG (2025). KNIME Analytics Platform — Documentation. Recuperado de: `https://docs.knime.com/`. Fecha de consulta: 16/03/2026.

[5] UCI Machine Learning Repository (1991). Wine Data Set. Recuperado de: `https://archive.ics.uci.edu/dataset/109/wine`. Fecha de consulta: 16/03/2026. Donantes: S. Aeberhard, D. Coomans y O. de Vel.